

Opening the Black Box: How Your Phone Finds Lost Devices

Nguyen Thai Binh

Bluetooth Low Energy (BLE) is one of the most pervasive wireless technologies in use today, embedded in virtually every smartphone, wearable, and a vast and growing population of low-power embedded devices. Overnight, Apple turned hundreds of millions of their devices into the infrastructure behind Find My - a crowd-sourced location-relay network - that allows owners to locate their lost devices even without working Internet connections.

Built on earlier works that reverse engineered the Offline Finding protocol used by AirTag, I developed `sendmy`, a one-way communication protocol that lets BLE-only devices transmit arbitrary data through the Find My network. This report discusses the learning journey and the protocol design. It also provides a proof-of-concept implementation on ESP32 hardware, and evaluates its behaviour in practice. It finally closes by discussing the open privacy and security limitations.

June 26, 2026

Contents

1. Introduction	3
1.1. Motivation	3
1.2. The Idea	3
1.3. Contributions	3
2. Background	4
2.1. Bluetooth Low Energy Advertising	4
2.2. Apple's Find My & Offline Finding Network	5
2.3. Related Work	5
3. Exploration & Methodology	6
3.1. Week 1: Understanding BLE Advertising	6
3.2. Week 2: The OF Packet Format & BLE Programming on ESP-IDF	6
3.3. Week 3: Packet Capture & Verification	6
3.4. Week 4 - 5: An AirTag Clone (First Milestone)	7
4. The sendmy Protocol	8
4.1. Design Goals & Layering	8
4.2. The Link Layer: Emitting an Accepted Beacon	8
4.3. The Carrier Layer: Encoding (and Recovering) a Message	9
4.4. Proof-of-Concept Implementation espsead	10
5. Evaluation	11
5.1. Does It Work End-to-End?	11
5.2. Throughput	11
5.3. Reliability	11
6. Limitations & Future Work	12
6.1. Privacy & Security	12
6.2. Hardware: Toward a Battery-Less Device	12
6.3. Characterising the Throughput Ceiling	12
6.4. Protocol Refinements	12
7. Conclusion	13
Appendix	14
Bibliography	15

1. Introduction

1.1. Motivation

Bluetooth Low Energy (BLE) is everywhere. It ships in virtually every modern device - from smartphones and tablets, to wearables and fitness trackers - and increasingly in the myriad of cheap, battery-powered embedded devices that make up the “Internet of Things”. The appeal is precisely that it is cheap and frugal: The 2.4 GHz BLE radio costs close to nothing and can run for months on a coin cell.

That frugality comes at a cost. BLE is a short-range technology, typically reaching only a few metres to a few tens of metres, and, unlike Wi-Fi or cellular, it provides no inherent route to the Internet. A device with *BLE-only* capability is therefore islanded: it can only talk to whatever happens to be nearby. For a whole class of low-cost, low-power devices, the absence of an uplink presents the single biggest barrier to being useful at a distance.

1.2. The Idea

Rather than adding an uplink (a SIM, a Wi-Fi module, a gateway), this project inquires whether an existing one can be **borrowed**. Apple’s *Find My* network is a crowd-sourced location service in which hundreds of millions of Apple devices silently listen for the BLE beacons emitted by lost items (such as AirTag), attach their own location, and upload an encrypted report to Apple on the lost item’s behalf. The owner later retrieves those reports to locate the item (more in Section 2.2).

The key observation is that this network will relay a beacon from **any** device that emits the right kind of advertisement; it does not check that the beacon came from a genuine, paired Apple accessory. Therefore, as long as a beacon broadcasts a valid advertisement that also embeds information, the device now owns a global uplink, albeit an inefficient and unreliable one.

1.3. Contributions

This report documents `sendmy`, my proof-of-concept realisation of that idea. Concretely, I contribute:

- A *layered protocol design* that separates the problem of “make a beacon the Find My network accepts” from the problem of “encode a message into that beacon”.
- A *working implementation* on commodity ESP32 hardware, including an end-to-end demonstration (`espsend`) and a precursor AirTag-style device (`esptag`) that established the relay channel was usable in the first place.
- An *evaluation* of what the channel can and cannot do - in particular its reliability and its fundamental throughput ceiling - and a discussion of the privacy and security work that remains.

2. Background

2.1. Bluetooth Low Energy Advertising

BLE devices communicate in one of two broad modes: *connection-oriented* exchanges between two paired devices, and *connection-less broadcasting*, in which a device simply emits short packets (“advertisements”) into the air for any listener to pick up. The Find My network (and this project) uses only the latter.

In BLE’s Generic Access Profile (GAP), a device that only broadcasts and never accepts connections plays the role of a **broadcaster**, and a device that only listens is an **observer**. An advertisement carries a small payload (on the order of a few dozen bytes) plus the advertiser’s device address. Crucially, that address can be a *random* address rather than a fixed hardware identifier, and the payload can include vendor-defined “manufacturer-specific” data. As we will see, both of these fields are where information can be encoded.

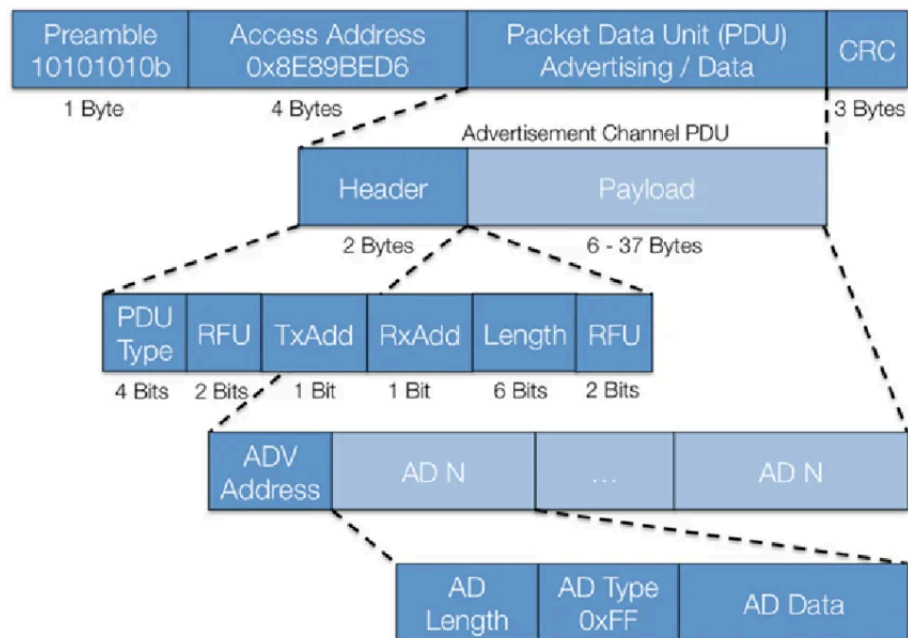


Figure 1: BLE Advertising Packet Structure [1]

Find My focuses only manipulating the Advertising Address (6-octet, MAC-like address) and the Advertising Data, and more specifically, the Manufacturer-Specific Data (BT_DATA_MANUFACTURER_DATA) type.

2.2. Apple’s Find My & Offline Finding Network

Apple’s **Find My** network - the part relevant here is sometimes called **Offline Finding (OF)** - lets a lost device be located even when it has no Internet connection of its own. The mechanism works roughly as follows:

1. A lost device continuously broadcasts a BLE advertisement containing a rotating public key.
2. Any nearby Apple device (“finder”) that hears the advertisement encrypts its own current location to that public key and uploads the encrypted report to Apple’s servers.
3. The owner of the lost device, who knows the corresponding private key, later downloads and decrypts the reports to recover the device’s location history.

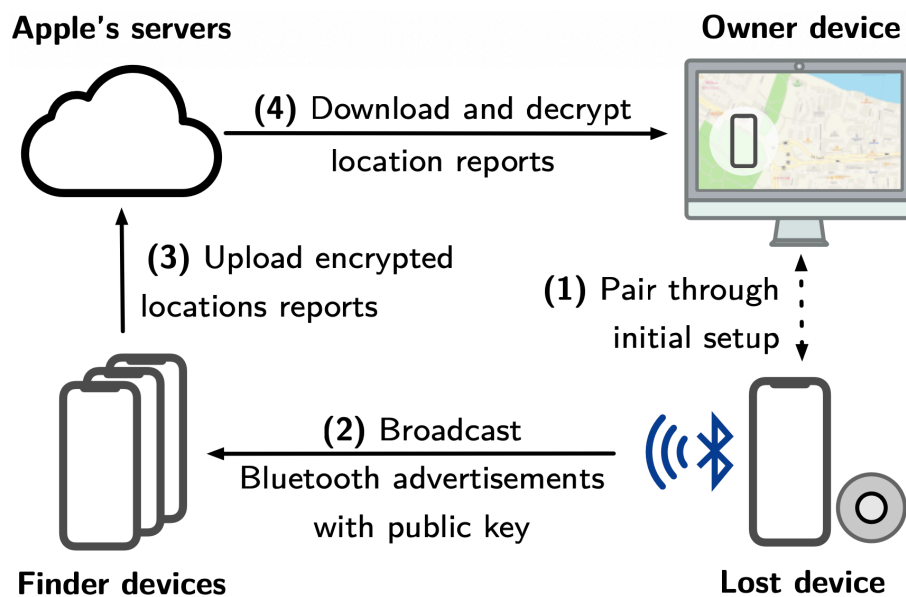


Figure 2: Simplified OF workflow [2]

Two properties of this design make it attractive to repurpose. First, the finder relays beacons *opportunistically and anonymously* - it does not authenticate the lost device. Second, because the location is encrypted to a key the finder cannot read, Apple’s network is content-agnostic about what it carries. The advertised “public key” is, from the network’s point of view, just an opaque blob that happens to be a valid point on a particular elliptic curve.

2.3. Related Work

The foundational demonstration that one can build one’s own Find My-compatible devices is shown by [2], which reverse-engineered the Offline Finding protocol and showed that arbitrary microcontrollers could participate in the network. Subsequent, more privacy-focused research [3] examined the protocol’s exposure and hardening. The insight that the *content* of the advertised key can be used as a data-carrying channel - turning a location protocol into a general-purpose uplink - is the conceptual starting point for `sendmy`.

In the process of implementing `sendmy`, I became aware of a same-name GitHub repository^o that essentially does the same thing, albeit with no security measures implemented.

3. Exploration & Methodology

This section documents the path I took to reach the final protocol, roughly in the order it happened. It is deliberately a narrative of *what I needed to learn, what I tried, and what each step let me do next* - the dead ends and verification steps are part of the contribution, because they are what gave me confidence the later protocol was sound.

3.1. Week 1: Understanding BLE Advertising

My first goal was simply to understand how BLE broadcasting works in practice, independent of Apple. I studied the GAP advertising model - broadcaster vs. observer roles, the structure of an advertising payload, and the distinction between connectable and non-connectable advertising. The practical takeaway was that a device can broadcast a custom payload and a custom (random) address without any pairing or connection, which is exactly the primitive the rest of the project would need.

The book *Getting Started with Bluetooth Low Energy* [4] has been an invaluable resource in this endeavour. Furthermore, the documentation for *NimBLE*, the BLE stack that I would eventually build on, was more complete and helpful than most embedded libraries out there.

3.2. Week 2: The OF Packet Format & BLE Programming on ESP-IDF

Next I turned to the specific format the Find My network expects, and to the toolchain I would build on. The Offline Finding advertisement is, structurally, a manufacturer-specific BLE record tagged with Apple's company identifier and an OF type byte; the bulk of its content is the rotating key. I worked out how the key is laid out across the advertising payload and the device address, so that a beacon I generated would be indistinguishable from one a genuine tag emits.

In parallel I came up to speed on *ESP-IDF*, Espressif's development framework for the ESP32, and its *NimBLE* port. The takeaway from this week was two-fold; **first**, I further familiarised myself with embedded programming, in aspects like the *FreeRTOS* threading model and hardware interrupts, and **second**, I had a complete picture of how the bytes are ordered in the advertising data section, as shown in the C code below.

```
typedef struct __attribute__((packed)) {
    uint8_t of_type;           // OF type
    uint8_t of_len;            // OF len
    uint8_t status;            // Status (battery, etc.)
    uint8_t key_mid[22];       // Bytes 6 - 27 of the advertising key
    uint8_t key_hi;            // Top 2 bits (of byte 0) of the key
    uint8_t hint;              // Unknown
} ble_adv_payload_t;
```

3.3. Week 3: Packet Capture & Verification

Knowing the format on paper is not the same as emitting it correctly. To verify my beacons byte-for-byte, I used *Wireshark* together with an *nRF52840* acting as a BLE sniffer to capture advertisements over the air. This let me compare my generated packets against the documented OF structure (and against real tag traffic) and catch discrepancies - byte order,

field lengths, the bits of the address that must be set a particular way - that would otherwise have caused silent failures downstream.

The takeaway from this step was the most important kind a systems project can get: *independent confirmation that my device was emitting a structurally valid OF advertisement*, not merely something I believed to be valid. Every later result rests on this verification.

3.4. Week 4 - 5: An AirTag Clone (First Milestone)

Before attempting to send *data*, I built a device that simply *worked as a tag*: `esptag`, an AirTag clone in everything but the official pairing. It broadcasts genuine Offline Finding beacons and runs the same kind of rotating-key scheme a real accessory uses, so that anyone holding its provisioned secrets can later fetch and decrypt the location reports the network collected for it.

`esptag` is not the data channel but a *beacon*, and it does not use the data-carrying layer at all. Its purpose in the project was to **prove that the relay channel was real and usable from my hardware**: if Apple's network would collect and return location reports for beacons my ESP32 emitted, then the network was accepting my advertisements end-to-end, and the channel I wanted to build a protocol on top of actually existed. Confirming this - fetching real location reports for a device I built - is what justified moving on to the data transmission protocol proper.

```
idf.py (python)
--- esp-idf-monitor 1.9.0 on /dev/cu.usbmodem101.115200
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H
ESP-ROM:esp32r3-20210327
Build:Mar 27 2021
rst:0x15 (USB_UART_CHIP_RESET),boot:0x8 (SPI_FAST_FLASH_BOOT)
Saved PC:0x4037cfa2
--- 0x4037cfa2: esp_cpu_wait_for_intr at /COMPONENT_ESP_HW_SUPPORT_DIR/cpu.c:64
SPW000ee
mode:010, clock div:1
load:0x3fccc2820, len:0x105c
--- 0x403c8700: _stext at 77:?
load:0x403c8700, len:0x2d74
entry 0x403c8888
--- 0x403c8888: call_start_cpu0 at /COMPONENT_MAIN_DIR/bootloader_start.c:27
I (176) main: tag provisioned, counter=0
I (176) ble_adv: initialising BLE advertising (rotate every 60000 ms)
I (216) ble_adv: nimble port initialised, starting host task
I (216) ble_adv: nimble host task started
I (216) ble_adv: nimble host synced
I (216) ble_adv: advertising as F7:35:05:7A:9F:C2 (counter=0)
D (226) ble_adv: 1e ff 4c 00 12 19 94 72 82 02 81 2f cd b3 2a 6b
D (226) ble_adv: 1b 26 87 73 64 2a 02 00 2b dc 45 74 b3 01 00
I (236) ble_adv: rotation armed, first epoch in 8000 ms
I (60236) ble_adv: epoch elapsed, rotating from counter=0
I (60286) ble_adv: advertising as FE:48:18:7A:65:5F (counter=1)
D (60286) ble_adv: 1e ff 4c 00 12 19 4f 2b 17 f5 26 3b 5e ff 43 b8
D (60286) ble_adv: 13 63 cd f6 07 f6 08 1c 91 4c df b4 c3 03 00
I (120286) ble_adv: epoch elapsed, rotating from counter=1
I (120336) ble_adv: advertising as C2:53:4F:81:24:18 (counter=2)
D (120336) ble_adv: 1e ff 4c 00 12 19 94 81 69 d2 dd a0 35 91 0f 94
D (120336) ble_adv: 46 f0 7f 4c 05 fc 98 54 87 d2 28 f4 02 00 00
I (180346) ble_adv: epoch elapsed, rotating from counter=2
I (180386) ble_adv: advertising as F0:80:18:7D:35:64 (counter=3)
D (180386) ble_adv: 1e ff 4c 00 12 19 ec 53 e8 34 5a 4e 67 a6 95
D (180386) ble_adv: 49 76 ce 2c c7 e5 bc 77 ab ad 8d 32 6e 03 00
I (240396) ble_adv: epoch elapsed, rotating from counter=3
I (240446) ble_adv: advertising as 06:10:12:19:00:15 (counter=4)
D (240446) ble_adv: 1e ff 4c 00 12 19 74 7e 60 35 94 ba ca 32 1b 46
D (240446) ble_adv: 1f c7 54 44 f0 e3 72 32 b5 a6 b3 eb 2d 01 00
I (300446) ble_adv: epoch elapsed, rotating from counter=4
I (300496) ble_adv: advertising as CF:18:6C:C4:CF:30 (counter=5)
D (300496) ble_adv: 1e ff 4c 00 12 19 04 2c 58 9a 83 fc 54 cc 74 b5
D (300496) ble_adv: 34 03 07 28 07 9c 97 00 93 a0 a5 e4 3f 03 00
I (360496) ble_adv: epoch elapsed, rotating from counter=5
I (360546) ble_adv: advertising as D0:EF:8F:05:84:49 (counter=6)
D (360546) ble_adv: 1e ff 4c 00 12 19 01 b0 ce d5 b6 a6 62 61 ea a0
D (360546) ble_adv: 1c 76 ee bd e4 c3 67 17 5f 98 07 41 be 01 00

scripts/venv/bin/python (python)
> scripts/venv/bin/python scripts/scan_findmy.py -d 0
Scanning for offline-finding advertisements (Ctrl-C to stop)...
[ 1] F7:35:05:7A:9F:C2 rssi=-38 (Separated)
p_curr = 7735d57a9fc2728202812fcd32a6b1b268773642a020b2bdc4574b3 <-- first identity
[ 2] FE:48:18:7A:65:5F rssi=-31 (Separated)
p_curr = fe48187a655f2b17f5263b5eff43b81363cdf8d7f6081c914cdfb4c3 <-- new epoch (rotation)
[ 3] C2:53:4F:81:24:18 rssi=-28 (Separated)
p_curr = 02534f8124181b692dda03591019446f07f4c05fc985487d2281402 <-- new epoch (rotation)
[ 4] E0:60:18:7D:35:64 rssi=-29 (Separated)
p_curr = e060187d356453e8345a4e0e67a6954976ce2cc7e5bc77abad8d326e <-- new epoch (rotation)
[ 5] C6:EB:51:A9:00:1E5 rssi=-29 (Separated)
p_curr = 06eb51a9000e57d6033940ac321b451c75444f0e37232b5a6b3eb2d <-- new epoch (rotation)
[ 6] CE:99:7F:6D:68:31 rssi=-27 (Separated)
p_curr = 0e997f6d6831092de1392ddc77fc65f6ca77d34e82e73495d66ae994 <-- new epoch (rotation)
[ 7] CF:18:6C:C4:CF:30 rssi=-27 (Separated)
p_curr = cf186cc4cf30c589a83fc54cc74b534038728079c970d93a0a5e43f <-- new epoch (rotation)
[ 8] D0:EF:8F:05:84:49 rssi=-29 (Separated)
p_curr = 50ef8f05b449b0ced5b6a6b261aa01c76eebde4c367175f980741be <-- new epoch (rotation)
[]

.X_C2P107(esptag -zsh)
> scripts/venv/bin/python scripts/fetch_reports.py --epochs 100
Derived 100 epoch keys (0..99): p.0 = 7735d57a9fc2728202812fcd32a6b1b268773642a020b2bdc4574b3.
Restored session for:
18 report(s) across 100 epoch(s) (oldest first):
[epoch 0] 2026-06-13T11:38 lat=+1.292 lon=+103.772 ±60m conf=0
[epoch 0] 2026-06-13T11:39 lat=+1.292 lon=+103.771 ±28m conf=0
[epoch 0] 2026-06-13T11:39 lat=+1.292 lon=+103.772 ±56m conf=0
[epoch 1] 2026-06-13T11:40 lat=+1.292 lon=+103.772 ±40m conf=0
[epoch 1] 2026-06-13T11:40 lat=+1.292 lon=+103.772 ±74m conf=0
[epoch 1] 2026-06-13T11:40 lat=+1.292 lon=+103.772 ±74m conf=0
[epoch 1] 2026-06-13T11:40 lat=+1.292 lon=+103.772 ±133m conf=0
[epoch 2] 2026-06-13T11:41 lat=+1.292 lon=+103.772 ±82m conf=0
[epoch 2] 2026-06-13T11:41 lat=+1.292 lon=+103.772 ±74m conf=0
[epoch 2] 2026-06-13T11:41 lat=+1.292 lon=+103.772 ±40m conf=0
[epoch 2] 2026-06-13T11:41 lat=+1.292 lon=+103.772 ±77m conf=0
[epoch 3] 2026-06-13T11:41 lat=+1.292 lon=+103.772 ±73m conf=0
[epoch 3] 2026-06-13T11:42 lat=+1.292 lon=+103.772 ±40m conf=0
[epoch 4] 2026-06-13T11:43 lat=+1.292 lon=+103.772 ±62m conf=0
[epoch 4] 2026-06-13T11:43 lat=+1.292 lon=+103.772 ±40m conf=0
[epoch 5] 2026-06-13T11:44 lat=+1.292 lon=+103.772 ±57m conf=0
[epoch 5] 2026-06-13T11:44 lat=+1.292 lon=+103.772 ±55m conf=0
[epoch 5] 2026-06-13T11:44 lat=+1.292 lon=+103.772 ±60m conf=0
```

Figure 3: Screenshot showing the debug information from ESP32 (left), BLE scanner script (top right), and location report fetcher (bottom right)

4. The `sendmy` Protocol

With the channel established, the core contribution of the project is `sendmy`: a protocol for sending *arbitrary data*, rather than *location*, through the same network.

4.1. Design Goals & Layering

I designed `sendmy` as a small layered stack, by analogy with the TCP/IP model, so that two largely independent concerns could be solved separately:

- A **link layer** (`sendmy_link`) responsible only for the question “*how do I emit a beacon the Find My network accepts, carrying a given block of bytes?*”
- A **carrier layer** (`sendmy_carrier`) responsible for “*how do I encode a message into that block, and how does the receiver recover it?*”

In this analogy, Apple’s Find My network itself plays the role of the network layer - the thing that actually relays bytes from one place to another - while my two components sit below and above it. The benefit of the split is that the link layer is reusable for any payload, and the carrier layer can be reasoned about (and changed) without touching the BLE details. In fact, the link layer had been implemented for `esptag` well before the carrier layer even came into existence, so it actually took a whole afternoon to decouple the code from the cryptographic core used by `esptag`.

4.2. The Link Layer: Emitting an Accepted Beacon

The link layer (`sendmy_link`) takes a fixed-size **28-byte block** and broadcasts it as a structurally valid Offline Finding advertisement. It is implemented as a thin wrapper around the NimBLE host stack, configured as a **non-connectable, non-discoverable broadcaster** - the device never accepts connections; it only emits advertisements at a configurable interval (constrained to the BLE-legal range of 20–10240 ms, internally expressed in the spec’s 0.625 ms units).

The defining job of the link layer is to scatter the 28 key bytes across two fields of the advertisement - the **manufacturer-specific payload** and the **BLE random device address** - in exactly the layout a genuine Apple tag uses, so that the network cannot distinguish my beacon from a real one.

The manufacturer payload. Offline Finding rides inside a BLE manufacturer-specific AD structure (AD type `0xFF`) tagged with Apple’s company identifier `0x004C`. `sendmy_link` reproduces the byte-for-byte structure of a genuine tag:

<code>1e ff 4c 00</code>	AD length + type <code>0xFF</code> + Apple company ID (<code>0x004C</code> , little-endian)
<code>12</code>	Offline Finding type byte
<code>19</code>	OF payload length = 25
<code>00</code>	status byte
<code><22 bytes></code>	key[6..27] (22 of the 28 key bytes)
<code><1 byte></code>	key[0] >> 6 (the two most-significant bits of key[0])
<code>00</code>	hint byte

This accounts for 23 of the 28 key bytes directly (`key[6..27]`), plus the top two bits of `key[0]` carried in a dedicated byte.

The random address. The remaining six bytes (`key[0..5]`) are carried in the 6-byte BLE random device address, written in **reversed byte order** (`key[5]` first, `key[0]` last). One constraint is imposed by BLE itself: for the controller to treat the address as a **static random address**, the two most significant bits of the top octet must be 1. `sendmy_link` therefore forces those two bits high - which is precisely why the carrier layer also emits the two MSBs of `key[0]` separately in the payload's dedicated byte, so the receiver can restore them.

Between these two fields a receiver can reconstruct all 28 bytes: 22 from the payload body, the top two bits of `key[0]` from the dedicated payload byte, and the remaining six (including the low bits of `key[0]`) from the re-reversed address.

The public interface is intentionally minimal - an initialiser (`sm_ll_init` , which brings up the NimBLE host task and fires a callback once the controller has synced) and a thread-safe `sm_ll_set_key` , which copies a new 28-byte block under the NimBLE mutex, stops the current advertisement, re-derives the random address, rebuilds the payload, and restarts. Each call to `sm_ll_set_key` therefore rotates the advertised identity - which is exactly how a stream of distinct messages is sent.

4.3. The Carrier Layer: Encoding (and Recovering) a Message

The carrier layer (`sendmy_carrier`) turns a message into the 28-byte block the link layer broadcasts. A message is small by design - a **single payload octet**, tagged with a **32-bit message ID** (`mid`) - and the encoding is bound to a **32-byte shared secret** (the **Unilink ID**, `uid`) held by both sender and receiver.

Derivation. From these three inputs the carrier layer computes the 28-byte block deterministically, using a keyed hash (HMAC-SHA256, keyed by `uid`) over the message ID and payload. Because the function is deterministic, sender and receiver who share `uid` will always agree on the block for a given (`mid` , `payload`) . Critically, the **payload is folded into the hash input and never appears on the wire** - an observer who lacks `uid` cannot derive the block for any payload, and cannot even tell which of the 256 possible payloads a given beacon encodes. The exact construction is given in Appendix A. on the device the hash runs through the PSA Crypto API, and every intermediate buffer is scrubbed from memory before the function returns, on both success and failure paths.

Why the block is a curve point, not just a hash. This is the central design problem, and also what makes this different from other implementations. A Find My advertising key is not an arbitrary 28-byte string - the network interprets it as a point on a specific elliptic curve (NIST P-224). A finder that hears the beacon reconstructs that point and uses it to encrypt its location report. A raw hash output is a valid point only about **half the time**; for the other half no such point exists, the finder cannot encrypt to it, and it files **no report at all** - the message is silently lost, with no error visible to the sender.

`sendmy_carrier` sidesteps this entirely. Instead of advertising the hash directly, it uses the hash as a **private key** and advertises the corresponding **public key**, which is a valid curve point **by construction**. Deliverability is therefore 100% rather than ~50%. The mathematics of this construction - the curve, the public-key derivation, and a negligible edge case handled by re-derivation - is deferred to Appendix A.

Implementation note. The elliptic-curve operation is written with the help of Claude (`p224.c`) because the cryptographic library shipped with ESP-IDF 6.0 no longer includes this particular curve. It might not seem like a good idea to ask an LLM to implement cryptographic primitives; however, in this case, the primitives are *relatively* simple to implement thanks to the special parameters of this particular curve, even though the underlying mathematics is largely beyond me. The field-arithmetic specifics are summarised in Appendix A.

Recovery by existence. The receiver never reads a payload off the wire; instead it recovers it by *probing which message the network relayed*. Holding the same `uid`, the receiver fixes the `mid` and enumerates all **256 candidate payloads** (`0x00` – `0xFF`), running each through the identical derivation above to produce 256 candidate carriers. It then computes the SHA-256 hash of each candidate - the same hash Apple’s network uses to index reports - and queries Apple’s location endpoint for all 256 at once. **The candidate whose hash returns a report reveals the byte that was actually transmitted.** The host-side receiver (`fetch_reports.py`) derives carriers with Python’s `cryptography` library and matches the firmware byte-for-byte, so the two ends stay in lockstep.

Because every carrier is a valid P-224 point by construction, the existence test is unambiguous: a candidate that returns **no** report means that byte was **not** sent (or its beacon was lost in transit), never that the carrier was an undeliverable point that the network silently dropped. This is what makes the “absence of a report $\Rightarrow$ transmission loss” interpretation relied on in Section 5 trustworthy.

4.4. Proof-of-Concept Implementation `espsend`

`espsend` wires the two layers together into an end-to-end demonstration on a Seeed XIAO ESP32-S3. On boot it loads the shared secret, then repeatedly: picks the next message, derives the carrier, hands it to the link layer, and advertises it for a fixed window before moving to the next. On the host side, a companion script recovers each transmitted byte by the existence-probing method of Section 4.3. The result is a complete, if slow, data pipe from a BLE-only device to a remote receiver, with Apple’s network as the only thing in between.

The code for `espsend`, as well as `esptag` and both layers’ IDF component, can be found in this GitHub repository^o.

5. Evaluation

5.1. Does It Work End-to-End?

I have yet to formalise a testing procedure, manual or automated, for the protocol. However, these are the steps I use to verify during development:

- Run the firmware on the ESP32 with a new Unilink ID generated;
- Run a BLE sniffer to verify that BLE packets are indeed being broadcast and in the correct format;
- Use the report fetcher to retrieve “location reports” directly from Apple. This report technically can be decrypted (since we have all parameters required to reconstruct the private key) to get the locations; however, that is not relevant. The existence of the report implies the payload being sent, as explained in Section 4.3.

The headline result is that the channel functions end-to-end: bytes emitted by the ESP32 are recoverable using receiver scripts. Because every carrier is a valid curve point by construction (Section 4.3), a successfully relayed message is recovered reliably.

5.2. Throughput

The channel is **low-throughput**, but it is important to be precise about *why*. The rate I currently observe is set by the firmware, not by a measured limit of the network. `espsend` transmits one payload byte at a time and advertises each byte for a fixed **60-second window** before rotating the key to the next message. That single design choice - one byte per minute - works out to roughly **0.13 bits per second**, and it is deliberately conservative: a long window gives Apple’s relay network ample opportunity to pick up the beacon and file a report, which keeps the demonstration reliable and easy to follow.

Crucially, this is an *artificial* rate, not a fundamental ceiling. Nothing stops the firmware from rotating the key more often and packing more bytes into the same wall-clock time. The open question is what that costs in reliability: a key that advertises for a shorter window has fewer opportunities to be observed and relayed, so I expect shortening the window to trade throughput against an increasing rate of dropped (never-relayed) bytes. Where the genuine ceiling lies - the rotation interval beyond which drops outpace the gain - is governed by how frequently the relay network actually samples nearby beacons, which I have **not yet characterised**. I therefore report 0.13 bits per second as the rate of the present configuration, and treat the true upper bound as an open question (see Section 6.3).

5.3. Reliability

Separately from rate, the *reliability* of a relayed message is high by design: the valid-curve-point construction removes the ~50% silent-loss mode that a naive encoding suffers, so the dominant remaining cause of a missing byte is simply that no finder happened to be in range during the advertising window.

Given the lack of acknowledgement in the design of BLE Legacy Advertising and the Offline Finding protocol, this problem most likely is unfixable.

6. Limitations & Future Work

6.1. Privacy & Security

The protocol is a proof of concept and is not yet secure against an adversary. Nevertheless, it incorporates relevant features to protect against eavesdroppers and Man-in-the-Middle attacks. However, there are still open attack vectors, several stemming from implementation hardware:

- **No protection against replay attacks.** There is no freshness mechanism, so a recorded beacon can be replayed by an observer. Due to the one-way nature of the protocol and the lack of reliable time storage, this is likely unfixable.
- **Secret key materials stored in plaintext.** The Unilink is flashed in storage in plaintext, so an attacker with physical access can easily dump the key. Hardening is possible but requires the use of flash encryption and/or a secure element, which is out of scope for this project.
- **Variable-time cryptographic primitives.** The current Claude-written implementation of EC cryptography uses a few naive algorithms that are vulnerable to timing attacks. This can be fixed by using vendored, audited cryptographic libraries.

Closing these gaps - moving toward a design that offers meaningful privacy against an active adversary - is the main direction for the protocol's future work.

6.2. Hardware: Toward a Battery-Less Device

The original long-term goal of the project, as proposed by the supervisor, is a **battery-less Offline Finding device** - one that harvests its energy and uses `sendmy` to report. The protocol's extremely low duty cycle and one-way nature make it a natural fit for energy harvesting, but realising it is a hardware effort that remains ahead.

6.3. Characterising the Throughput Ceiling

As Section 5.2 notes, the present 0.13 bits per second is an artefact of the firmware's conservative 60-second key-rotation window, not a measured limit. The most immediate piece of future work is to find the *real* ceiling. Concretely, I plan to sweep the rotation interval downward - advertising each key for progressively shorter windows - and, for each setting, measure the fraction of bytes that are successfully relayed versus silently dropped. This should reveal the trade-off curve between rotation frequency and reliability, and locate the crossover point beyond which faster rotation loses more bytes than it gains.

The outcome would also illuminate the underlying behaviour of the relay network: how often, on average, a beacon must be advertised before some finder samples and reports it. That sampling cadence is the true throughput governor, and pinning it down is a prerequisite for any serious claim about how fast `sendmy` can be pushed.

6.4. Protocol Refinements

Beyond security, there is room to extend the carrier layer with richer message framing, error handling for lost bytes, and acknowledgement-free reliability techniques suited to a one-way, lossy channel.

7. Conclusion

This project set out to give a BLE-only device something it fundamentally lacks - a way to reach beyond its few metres of range - without adding any hardware to do so. By treating Apple's Find My network as a borrowed, crowd-sourced uplink and disguising data as the beacons of a lost device, `sendmy` achieves exactly that: a working, one-way data channel from commodity ESP32 hardware to a remote receiver, with no Internet connection on the device itself. The channel is slow and not yet secure, but it is real and reliable within its limits, and it demonstrates that the ubiquity of BLE plus the ubiquity of opportunistic relay networks can combine into reach that neither offers alone. The path from here runs through the protocol's security and toward battery-less hardware.

Appendix

A. Cryptographic Construction

A.1. Carrier Derivation

The 28-byte block is produced by a construction modelled on a single HKDF-Expand output block:

```
info    = "smv1" || mid (big-endian, 4 bytes) || payload || attempt
block   = HMAC-SHA256(key = uid, info || 0x01)[:28]
d       = int(block, big-endian)
carrier = X(d · G)
```

The HMAC binds all three inputs (uid , mid , payload). The "smv1" tag is a protocol-version label, and attempt supports the rejection sampling described below.

A.2. The Valid-Point Construction

A Find My advertising key is the **x-coordinate of a NIST P-224 (secp224r1) public key**. A finder reconstructs the full curve point and performs an Elliptic-Curve Diffie-Hellman step to encrypt its location report to it. A raw truncated hash is a valid x-coordinate only about half the time; for the remaining half no point exists on the curve, so the finder cannot encrypt and files no report - a silent loss.

sendmy_carrier avoids this by interpreting the derived block as a **private scalar d** and advertising $X(d \cdot G)$ - the affine x-coordinate of the public key $d \cdot G$, where G is the curve's base point. As a public key this is **always a valid P-224 point**, giving 100% deliverability.

The one constraint is that d must lie in $[1, n-1]$, where n is the order of the curve. If the derived block falls outside that range, the 1-byte attempt counter is incremented and the block is re-derived (rejection sampling). The probability of a single rejection is on the order of 2^{-112} , so in practice attempt is always 0.

A.3. Field Arithmetic (p224.c)

The secp224r1 scalar multiplication is Claude-assisted hand-written because mbedTLS/PSA in ESP-IDF 6.0 no longer ships secp224r1. It uses fixed 7-word (224-bit) integer arithmetic:

- A schoolbook multiply feeding the **special NIST P-224 fast reduction**: the prime $p = 2^{224} - 2^{96} + 1$ admits reduction by additions and subtractions alone, with no division.
- A **Jacobian double-and-add** ladder for the scalar multiplication $d \cdot G$, avoiding per-step modular inversions.
- A **single modular inverse** at the end, to convert the final Jacobian point back to affine coordinates and recover the x-coordinate.

This costs roughly 100 ms per carrier on the ESP32-S3. The host-side receiver performs the identical derivation with Python's cryptography library, so the two ends produce byte-identical carriers.

Bibliography

- [1] A. Basalamah, 'Oppotunistic crowdsensing framework for Internet of things using Bluetooth low energy', 2016. doi: 10.1049/joe.2016.0062^o.
- [2] A. Heinrich, M. Stute, T. Kornhuber, and M. Hollick, 'Who Can Find My Devices? Security and Privacy of Apple's Crowd-Sourced Bluetooth Location Tracking System', 2021. doi: 10.2478/popets-2021-0045^o.
- [3] H. Alamleh and A. Cantelli-Forti, 'Defending against BLE-based covert channels in crowdsourced location networks', 2026. doi: 10.1016/j.cose.2026.104835^o.
- [4] K. Townsend, C. Cufi, Akiba, and R. Davidson, *Getting Started with Bluetooth Low Energy*. O'Reilly Media, 2014. [Online]. Available: <https://www.oreilly.com/library/view/getting-started-with/9781491900550/>^o